Java Constructors — Practice MCQ Set 2

Topic: Constructors, Overloading, 'this' keyword | Attempt first, check answers at the end.

Q1. Which of these is NOT a valid rule for a constructor in Java?

A. It must have the same name as the class

B. It can have a return type of void

C. It runs automatically when an object is created with 'new'

D. It can be overloaded

Q2. Look at this class:

```
public class Item {
    String name;
    int qty;

    Item() {
        this.name = "Unnamed";
    }
}
```

A. name = null, qty = 0

B. name = "Unnamed", qty = 0

C. name = "Unnamed", qty = null

D. Compilation error

Q3. How many constructors can a single Java class have?

A. Exactly 1

B. Exactly 2 (default + parameterized)

C. As many as needed, as long as parameter lists differ

D. 0, constructors are optional and never allowed together

Q4. Predict the output:

```
public class Box {
    int size;
    Box(int size) {
        size = size;   // note: no 'this.'
    }
    public static void main(String[] a) {
        Box b = new Box(10);
        System.out.println(b.size);
    }
}
```

A. 10

B. 0

C. Compilation error

D. null

Q5. Which line correctly calls the 2-parameter constructor ProductClass(String productName, String description)?

A. new ProductClass();

B. new ProductClass("Mouse", "Wireless mouse");

C. new ProductClass("Mouse", 500, "Wireless", 10);

D. new ProductClass(500, "Wireless");

Q6. True or False: If a class already has a parameterized constructor, Java will still silently add a no-argument default constructor for you.

A. True, always

B. False — you must write it yourself if you need it

C. True, but only for public classes

D. False, Java never allows a no-arg constructor

Q7. What is printed?

```
public class Person {
    String name;
    Person(String name) { this.name = name; }
    Person() { this("Guest"); }

    public static void main(String[] a) {
        Person p = new Person();
        System.out.println(p.name);
    }
}
```

A. null

B. Guest

C. Compilation error

D. Person

Answer Key & Explanations

Q1. Correct Answer: B

A constructor never has a return type — not even void. Option B is the false statement.

Q2. Correct Answer: B

The constructor explicitly sets name to "Unnamed". qty is untouched, so it keeps int's default value, 0.

Q3. Correct Answer: C

A class can have any number of constructors via overloading, as long as each has a distinct parameter list.

Q4. Correct Answer: B

Without 'this.', 'size = size;' just assigns the parameter to itself. The field b.size is never touched, so it stays at int's default, 0. This is the classic 'this' keyword trap.

Q5. Correct Answer: B

Two String arguments in order (name, description) match the 2-parameter constructor exactly.

Q6. Correct Answer: B

Java only auto-supplies a default constructor when the class defines NO constructor at all. Once you write any constructor, that free default one disappears.

Q7. Correct Answer: B

this("Guest") calls the 1-parameter constructor from the default one (constructor chaining), so name becomes "Guest".